

SOLUTION SET

INDIAN INSTITUTE OF TECHNOLOGY (INDIAN SCHOOL OF MINES), DHANBAD

Winter End-Semester Examination, Session 2024-25

Examination: **II UG (Section: A, B, C, D)**

Time: **2 Hours**

Subject: **Computer Programming (NCSV101)**

Max. Marks: **50**

Q. No.	Question
1. (a)	<pre>#include<stdio.h> #include<stdlib.h> int main(){ FILE *fp; int i, j =0, k=0, n = 5, b[5], odd[10], even[10]; if((fp = fopen("input.txt", "r")) == NULL){ printf("Error! File not opened\n"); exit(0); } i = 0; while(fscanf(fp, "%d", &b[i]) != EOF){ printf("b[%d] = %d, ", i, b[i]); i++; } fclose(fp); for(i = 0; i < 5; i++){ if(b[i]%2 == 0){ even[j] = b[i]; j++; } else { odd[k] = b[i]; k++; } } if((fp = fopen("odd.txt", "w")) == NULL){ printf("Error! File not opened\n"); exit(0); } for(i=0; i < k; i++){ fprintf(fp, "%d ", odd[i]); } }</pre>

```

    fclose(fp);
    if((fp = fopen("even.txt", "w")) == NULL){
        printf("Error! File not opened\n");
        exit(0);
    }
    for(i=0; i < j; i++){
        fprintf(fp, "%d ", even[i]);
    }
    fclose(fp);
    return(0);
}

```

Evaluation Scheme:

- Only reading elements from file: 2 Marks
- Identification of odd and even, writing to file: 4 Marks
- 1 mark is deducted if error checking after opening a file is not done.
- Further marks are deducted for wrong function name or wrong function arguments.

- (b) Since each char takes 1 byte, int takes 4 bytes, and float takes 4 bytes, the size of model[15] array is 15bytes, size of price_in_lakhs is 4 bytes, size of structure variable p is 16 bytes. Since c1 is an union variable, size of c1 is 16 bytes.

Evaluation Scheme:

- Correct answer: 1 Mark
- Proper reason: 1 Mark

- (c)

```

#include<stdio.h>
struct point{
    float x;
    float y;
};

int main()
{
    struct point p[100];
    int n, i;
    float min_x, max_y;
    printf("Enter the value of n \n");
    scanf("%d", &n);
    for (i = 0; i < n; i++){
        printf("Enter point %d\n",i);
        scanf("%f %f", &p[i].x, &p[i].y);
    }

```

```

min_x = p[0].x;
max_y = p[0].y;
for (i = 1; i < n; i++){
    if(min_x > p[i].x){
        min_x = p[i].x;
    }
    if(max_y < p[i].y){
        max_y = p[i].y;
    }
}
printf("left most points: ");
for (i = 0; i < n; i++) {
    if (p[i].x == min_x){
        printf("(%f, %f), ", p[i].x, p[i].y);
    }
}
printf("top most points: ");
for (i = 0; i < n; i++) {
    if (p[i].y == max_y){
        printf("(%f, %f), ", p[i].x, p[i].y);
    }
}
}

```

Evaluation Scheme:

- Defining the structure: ½ Mark
- Taking correct input: ½ Mark
- Finding topmost: 1½ Marks
- Finding leftmost: 1½ Marks
- Handling the multiple topmost and leftmost: 1 mark
- Marks deducted for syntax error, wrong initialization.

Question 2(a) i: (3 marks)

Aspect	Arrays	Pointers	Structures
Basic Definition	Collection of elements of the same data type .	Variable storing the memory address of another variable.	Collection of different data types grouped under a single name.
Initialization	Can be initialized at declaration: <pre>int arr[3] = {1, 2, 3}; char str[] = "hello";</pre>	Initialized to an address, typically of another variable or dynamically allocated memory: <pre>int x; int *ptr = &x; int *arr = malloc(sizeof(int)*5);</pre>	Members initialized individually or using an initializer list: <pre>struct Person {int age; char name[20];}; struct Person p = {25, "Alice"};</pre>
Initialization After Declaration	Cannot directly reinitialize whole array, must assign individually: <pre>arr[0] = 5;</pre>	Can be reassigned easily: <pre>ptr = &y; or ptr = malloc(...).</pre>	Individual structure members can be set after declaration: <pre>p.age = 30; strcpy(p.name, "Bob");</pre>

Question 2 (a) ii: (3 marks)

(1) Nesting Structures within Structures (Valid)

In C, it is common and valid to nest structures within other structures. This is typically done to create complex data types.

Example:

```
#include <stdio.h>

struct Date {
    int day;
    int month;
    int year;
};

struct Person {
    char name[50];
    int age;
    struct Date birthDate; // Nested structure
};

int main() {
    struct Person p1 = {"Alice", 25, {12, 3, 2000}};
    printf("Name: %s, Age: %d, Birth Date: %d/%d/%d\n",
           p1.name, p1.age, p1.birthDate.day, p1.birthDate.month,
           p1.birthDate.year);
    return 0;
}
```

Output:

Name: Alice, Age: 25, Birth Date: 12/3/2000

(2) Nesting Functions within Functions (Invalid in Standard C)

Standard C **does not support nesting functions**. You cannot define a function within another function.

Invalid Example:

```
#include <stdio.h>

void outerFunction() {
    void innerFunction() { // INVALID in C
        printf("Inner function called.");
    }

    innerFunction();
}

int main() {
    outerFunction();
    return 0;
}
```

This will cause a compilation error like:

```
error: nested functions are not supported
```

In C, **function calls can be nested**, but **function definitions cannot be nested**.

☒ Valid: Nested Function Calls

You can call a function within another function call, like this:

```
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

int square(int x) {
    return x * x;
}

int main() {
    int result = square(add(2, 3)); // Valid nested call
    printf("%d\n", result); // Output: 25
    return 0;
}
```

Question 2 b:

```
#include <stdio.h>
#include <string.h>

// Given structure definition
struct Student {
    char name[50];
    int age;
    float marks;
};

// (i) Function to display student details (structure passed by value) (2 marks)
void displayStudent(struct Student s) {
    printf("Student Details:\n");
    printf("Name: %s\n", s.name);
    printf("Age: %d\n", s.age);
    printf("Marks: %.2f\n", s.marks);
}

// (ii) Function to update student's marks (structure passed by pointer) (3 marks)
void updateMarks(struct Student *s) {
    s->marks += 5.0; // adding 5 bonus points
}

(1 marks)
int main() {
    // Initializing a Student structure
    struct Student student1;

    // Assign values
    strcpy(student1.name, "Alice");
    student1.age = 20;
    student1.marks = 85.0;

    // Display initial details
    displayStudent(student1);

    // Update marks with bonus
    updateMarks(&student1);
}
```

```
printf("\nAfter adding bonus marks:\n");  
// Display updated details  
displayStudent(student1);  
  
return 0;  
}
```

Marking Scheme:

Question 2a:

- i) 3 marks: complete; 1.5: Partially correct; 1 marks: incomplete
- ii) 3 marks: complete; 1 marks: only example given, 0.5: only description given

Question 2b:

- i) 2 marks: complete code, 1 marks: partially incomplete; 0: wrong
- ii) 3 marks: complete code, 1 marks: incomplete; 0: wrong
- iii) 1 marks : complete; 0.5: incomplete, 0: wrong

3. (a)

(i) `*p = &b;`

This is an error. This attempts to assign the address of b (which is `&b`, an `int*`) to the value pointed to by p (which is `*p`, an `int`). `*p` is an integer value, and you can't store a pointer address in an integer variable.

(ii) `q = &*a;`

This is not an error. The expression `&*a` is equivalent to `&a`. The `&` operator returns the address of a variable, and the `*` operator dereferences a pointer (i.e., retrieves the value at that address). So, applying `&` to `*a` (which is the value of `a`) is the same as applying `&` to `a` directly.

(iii) `a = q - p;`

This is not an error. It is common practice to subtract the addresses of two pointers and the result is an integer. The result is the number of elements between them, not the actual memory locations. If `q` and `p` point to an array, then `q - p` will return the number of elements between the two pointers.

(iv) `p = &q;`

This is an error. `p` is declared as a pointer to an integer (`int*`). `&q` would give you the address of `q`, which is a pointer to an integer. You cannot store the address of a pointer in a variable that is declared to hold the address of an integer.

Evaluation Scheme:

- Correct answer with proper explanation: 1 Mark each
- Correct answer without or wrong explanation: 0 Mark each
- Completely wrong answer: 0 Mark each

(b) `typedef struct`

```
{  
    float x;          // x-coordinate  
    float y;          // y-coordinate  
}Point;
```

```
Point Midpoint_Cal(Point *p1, Point *p2)  
{  
    Point midpoint;  
    // Calculate the midpoint using the formula  
    midpoint.x = (p1->x + p2->x) / 2.0;  
    midpoint.y = (p1->y + p2->y) / 2.0;  
    return midpoint;  
}
```

Evaluation Scheme:

- Structure definition of a point: 1 Mark
 - Fully correct: 1 Mark

- Partially correct: ½ Mark
- Completely wrong: 0 Mark
- Definition of mid-point calculation function: 3 Marks
 - Fully correct: 3 Marks
 - Correct, but midpoint calculation is wrong: 2 marks
 - Correct, but has not returned value: 2 Marks
 - Partially correct: 1 Mark
 - Completely wrong: 0 Mark

(c)

```
#include <stdio.h>
int main()
{
    FILE *fp ;
    int n, sum=0, count=0 ;
    float avg ;
    fp=fopen("DATA", "r") ;           /* Open the file "DATA" in read mode */
    if(fp != NULL)
    {
        while(feof(fp) == 0)          /* I/O error handling
        {
            fscanf(fp, "%d", &n) ;      /* Read operation */
            sum = sum+n ;
            count++ ;
        }
    }
    fclose(fp) ;                      /* Close the file "DATA" */
    avg = (float)sum/count ;           /* Calculation of average value */
    fp = fopen("DATA", "a") ;          /* Open the file "DATA" in append mode */
    if(fp != NULL)
    {
        fprintf(fp, "%f", avg) ;       /* Append the average value at the end of the file */
    }
    fclose(fp) ;                      /* Close the file "DATA" */
    return 0 ;
}
```

Evaluation Scheme:

- Open the file in read mode: ½ Mark
- Read data from the file with error handling: 2½ Marks
 - Fully correct: 2½ Marks
 - Partially used error handling function: ½ Mark
 - Only written read operation: 1 Mark
 - Not used library function for error handling: ½ Mark
 - Completely wrong: 0 Mark
- Computation of average: 1 Marks
 - Fully correct: 1 Marks
 - Partially correct: ½ Marks
 - Completely wrong: 0 Mark

	- Append the average value to the file: 1 Mark - Fully correct: 1 Mark - Partially correct: ½ Mark - Completely wrong: 0 Mark
4. (a)	Pre: 5 Pre: 3 Pre: 1 Base(-1) Post: 1 Base(-2) Post: 3 Base(0) Post: 5 Pre: 2 Base(0) Post: 2 Base(-1) <u>Evaluation Scheme:</u> - Initial Descent and First Base Case: 1 Mark - First Branch Post Calls and Intermediate Base Cases: 1 Mark - Return to Top Level and Second Branch Initiation: 1 Mark - Second Branch Execution: 1 Mark
(b)	<pre>#include <stdio.h> #include <stdlib.h> int main(void) { /* Given arrays */ int dataSet1[] = {10, 20, 30, 40}; int dataSet2[] = {5, 15, 25, 35, 45, 55}; /* 1. Determine lengths */ int len1 = sizeof dataSet1 / sizeof *dataSet1; int len2 = sizeof dataSet2 / sizeof *dataSet2; /* 2. Find the shorter length */ int shorterLen = (len1 < len2 ? len1 : len2); /* 3. Allocate exactly enough space */ int *elementSumArr = malloc(shorterLen * sizeof *elementSumArr); if (elementSumArr == NULL) { printf("Memory allocation failed\n"); return 1; } }</pre>

```

/* 4. Populate using only pointer arithmetic */
for (int i = 0; i < shorterLen; ++i) {
    *(elementSumArr + i) = *(dataSet1 + i) + *(dataSet2 + i);
}
/* 5. Print the results */
printf("Element-wise sums: ");
for (int i = 0; i < shorterLen; ++i) {
    printf("%d ", *(elementSumArr + i));
}
printf("\n");

/* 6. Clean up */
free(elementSumArr);
return 0;
}

```

Evaluation Scheme:

- Correctly calculating the lengths of dataSet1 and dataSet2 (e.g., using sizeof(arr)/sizeof(arr[0])) AND correctly determining the minimum (shorterLen) of these two lengths: 1 Mark
- Correctly using malloc to dynamically allocate memory for elementSumArr based exactly on the shorterLen calculated. Must allocate shorterLen * sizeof(int) bytes. Includes a check for malloc returning NULL (failure): 1 Mark
- Implementing a loop that correctly iterates from index 0 up to (but not including) shorterLen: 1 Mark
- Inside the loop, correctly calculating the sum of corresponding elements from dataSet1 and dataSet2 AND storing the result in the correct position in elementSumArr, using only pointer arithmetic for accessing elements of all three arrays during the summation/assignment (e.g., *(elementSumArr + i) = *(dataSet1 + i) + *(dataSet2 + i);). No marks awarded for this point if array bracket notation ([]) is used for accessing elements within the summation calculation: 1 Mark
- Printing Results and Memory Management: 1 Mark

(c) The code as written is **incorrect**. The single, critical issue is:

- **Returning the address of a local variable**
In configureSensor(), sensorId is a stack-allocated (automatic) variable. When the function returns, its stack frame is torn down and sensorId no longer exists, so returning &sensorId yields a **dangling pointer**. Dereferencing that pointer in main() invokes undefined behavior.

Example of what one might see (but cannot rely on):

Configuring sensor...

Function scope: sensorId (42)

Configuration done. Pointer received: 42

	Because the pointer refers to invalid memory, the actual result could be garbage or even a crash.
--	---

	<u>Evaluation Scheme:</u>
--	----------------------------------

- | | |
|--|--|
| | <ul style="list-style-type: none">• Correct explanation of main issues: 3 Marks• Incorrect answer: 0 Mark |
|--|--|